Plan de Ejecución Ágil y Gobierno Técnico para el Desarrollo de un Producto Software en 10 Semanas

La entrega eficiente de productos de software comerciales bajo restricciones temporales fijas y presupuestos patrocinados exige un rigor operativo elevado, un gobierno de desarrollo ágil transparente y la integración continua de prácticas de seguridad y calidad desde el primer día. El presente informe establece la estrategia detallada de ingeniería de software para ejecutar un proyecto de diez semanas mediante la metodología Scrum, estructurado en ciclos de entrega semanales (Sprints de 1 semana)[1]. El modelo operativo está diseñado para maximizar el retorno de inversión del cliente patrocinador, garantizando visibilidad continua del avance, gestión estricta del gasto energético del equipo y la entrega de un incremento funcional, probado y seguro al finalizar cada ciclo[2].

Estrategia de Gobierno Organizacional y Gestión de Roles Duales

El equipo de desarrollo está compuesto por cinco ingenieros de software con responsabilidades distribuidas. En equipos reducidos, la asignación de roles duales es inevitable; sin embargo, para prevenir conflictos de interés, sobrecarga cognitiva y degradación en la toma de decisiones, se implementan salvaguardas explícitas de gobierno[4].
La superposición del rol de Product Owner y Developer en Amir presenta el riesgo inherente de favorecer atajos técnicos en beneficio de características de negocio o, inversamente, priorizar refactorizaciones complejas que no aportan valor inmediato al cliente patrocinador[5]. Para mitigar este conflicto, las decisiones de priorización del backlog se gestionan formalmente en Notion mediante la trazabilidad del valor comercial, mientras que el esfuerzo se mide exclusivamente a través de estimaciones relativas consensuadas por todo el equipo[2].
Por su parte, la combinación de Scrum Master y Developer en José requiere que la facilitación del proceso prevalezca durante la eliminación de impedimentos, garantizando que el equipo no sea interrumpido por solicitudes externas durante el Sprint[4]. La autonomía de calidad recae en Miguel como QA, quien posee la autoridad procesal para bloquear la transición de cualquier historia hacia el estado de completado si no supera las suites de pruebas automatizadas[8].

Integrante del Equipo	Rol Primario	Rol Secundario	Responsabilidades Clave de Gobierno	Mecanismo de Mitigación de Riesgo por Rol Dual

Amir	Product Owner (PO)	Developer	Priorización del Product Backlog, interlocución con el cliente patrocinador, definición de Criterios de Aceptación, desarrollo Java Backend[4].	Separación estricta entre la visión de negocio y la estimación técnica; delegación del arbitraje de esfuerzo al equipo durante el Sprint Planning para evitar el sesgo de autoasignación [5].
José	Scrum Master (SM)	Developer	Facilitación de ceremonias Scrum, eliminación de bloqueos, monitoreo de velocidad, desarrollo Java Backend[4].	Mantenimiento de la neutralidad procesal durante las retrospectivas; intervención estricta ante desviaciones de alcance solicitadas a mitad de Sprint[5].
Adolfo	Developer	N/A	Diseño de arquitectura de software en Java, implementación de APIs REST, persistencia de datos y pruebas unitarias.	Enfoque exclusivo en la velocidad de desarrollo del backend y la adherencia a estándares de codificación limpia.

Abel	Developer	N/A	Desarrollo de componentes backend Java, integración de servicios de terceros y optimización de consultas a la base de datos.	Enfoque continuo en la ejecución técnica y refactorización para prevenir la acumulación de deuda técnica[9].
Miguel	Developer	Quality Assurance (QA)	Desarrollo de funcionalidad Java, automatización de pruebas de interfaz y API mediante Katalon Studio, verificación de la Definición de Hecho[8].	Autonomía de QA para rechazar incrementos que no cumplan con la suite de pruebas automatizadas, garantizando que el rol de Developer no comprometa los criterios de aceptación.

Arquitectura Tecnológica y Ecosistema de Herramientas

La selección del ecosistema tecnológico responde a la necesidad de construir una arquitectura robusta, escalable y mantenible en Java, respaldada por un pipeline de integración continua que automatiza la calidad del código, la aseguración de interfaces y la seguridad ofensiva[8].
La capa central de backend y APIs RESTful se construye sobre Java, garantizando un rendimiento tipado y desacoplado. Cada cambio de código fuente subido al repositorio activa automáticamente la fase de compilación e inspección estática en SonarQube, la cual audita reglas OWASP, detecta vulnerabilidades, fallos de seguridad (*bugs*) y mide la cobertura de pruebas unitarias[8].
Una vez que el backend supera las barreras de calidad estática, el software se despliega en el entorno de pruebas (*staging*), donde Katalon Studio ejecuta pruebas funcionales y de regresión automatizadas sobre la interfaz de usuario y las interfaces de programación[8].

Contemporáneamente, el módulo de seguridad dinámica ejecuta pruebas de penetración automatizadas mediante sqlmap directamente contra las APIs expuestas para certificar que no existan vectores de inyección SQL en la capa de persistencia[12].
Toda la gestión del trabajo, el control de artefactos y la visibilidad para el cliente patrocinador se centralizan en una base de datos de Notion configurada en vista de tablero Kanban, mientras que WhatsApp se utiliza como un canal secundario exclusivo para alertas inmediatas y notificación de bloqueos críticos[2].

Herramienta	**Dominio Técnico**	**Fase de Aplicación**	**Integración en el Pipeline y Gobernanza**
Java	Lenguaje Core Backend	Desarrollo continuo	Construcción de arquitecturas desacopladas mediante APIs RESTful, gestión de transacciones y lógica de negocio.
SonarQube	Calidad de Código y SAST	Integración Continua (CI)	Análisis estático automatizado en cada compilación para validar barreras de calidad (*Quality Gates*), cobertura de código, bugs y vulnerabilidades OWASP[8].
Katalon Studio	Aseguramiento de Interfaz / QA	Validación de Incremento	Ejecución automatizada de pruebas funcionales de API y flujos de usuario (UI) al cierre del desarrollo de cada historia[8].
sqlmap	Seguridad Ofensiva	Auditoría de	Pruebas dinámicas

	(DAST)	Seguridad	de penetración contra los endpoints de la API Java para detectar y corregir inyecciones SQL antes de cada demostración[12].
Notion (Base de Datos / Tablero)	Gestión de Proyecto	Operación diaria	Gestión transparente del Product Backlog, Sprint Backlog, Criterios de Aceptación, registro de impedimentos y métricas de velocidad[2].
WhatsApp	Comunicación Operativa	Asíncrona / Inmediata	Canal de alertas críticas, coordinación de bloqueos urgentes y notificaciones automáticas de builds fallidas o fallos de seguridad[7].

Para garantizar que los endpoints Java no expongan vulnerabilidades a nivel de persistencia de datos, el equipo utiliza sqlmap de forma automatizada mediante scripts de prueba dinámicos, inyectando vectores de ataque controlados sobre los parámetros HTTP y encabezados de autorización (--headers="Authorization: Bearer <token>", --auth-type, --level=3) para certificar la resistencia del sistema[12].

La gestión del trabajo se centraliza en Notion mediante una Base de Datos configurada en vista de Tablero Kanban con estados estrictos: *Backlog Priorizado*, *Ready for Sprint* (cumpliendo la Definición de Listo), *In Progress*, *In Code Review / SonarQube*, *In QA (Katalon)*, *Security Audit (sqlmap)* y *Done* (cumpliendo la Definición de Hecho)[2]. WhatsApp se reserva exclusivamente para comunicaciones operativas de alta prioridad y alertas inmediatas, evitando la dispersión de decisiones de diseño o requisitos que deben quedar documentados formalmente en Notion[2].

Cadencia Operativa de Sprints Semanales (Sprints de 1 Semana)

Adoptar una cadencia de Sprints de una semana implica maximizar la agilidad y la capacidad de respuesta ante cambios, proporcionando al patrocinador una visibilidad casi continua del avance[1]. Sin embargo, este ritmo impone una sobrecarga de reuniones (*overhead*) que, si no se gestiona mediante un timeboxing riguroso, puede reducir drásticamente el tiempo efectivo de desarrollo[16]. En un Sprint de cinco días laborables, la pérdida imprevista de un solo día reduce la capacidad operativa del equipo en un 20%[16].

Evento Scrum	**Duración Máxima (Timebox)**	**Programación Sugerida**	**Objetivo Operativo en Cadencia Semanal**
Sprint Planning	90 minutos	Lunes (08:30 - 10:00)	Selección del objetivo del Sprint (*Sprint Goal*), revisión de Criterios de Aceptación e inclusión de historias preparadas en Notion[2].
Daily Standup	10 a 15 minutos	Martes a Viernes (08:30 - 08:45)	Sincronización diaria enfocada en tres preguntas clave: qué se completó, qué se hará hoy y qué impedimentos existen[2].
Backlog Refinement	45 minutos	Miércoles (16:00 - 16:45)	Estimación y descomposición de historias para el Sprint subsiguiente, evitando cuellos de botella en la planificación[7].

Sprint Review & Demo	45 a 60 minutos	Viernes (15:00 - 16:00)	Demostración en vivo al cliente patrocinador del software funcional desplegado en el entorno de staging[1].
Sprint Retrospective	30 a 45 minutos	Viernes (16:15 - 17:00)	Análisis del proceso de ingeniería, revisión de métricas de calidad y definición de un plan de acción inmediato para el siguiente Sprint[3].

Para amortiguar la fricción de la cadencia semanal, las ceremonias de planificación se basan en un trabajo previo de refinamiento continuo (*Backlog Refinement*) realizado a mitad de semana[7]. El Sprint Planning del lunes inicia con historias que ya cumplen plenamente la Definición de Listo (DoR)[2]. Asimismo, la demostración de los viernes no utiliza diapositivas ni entornos simulados; se ejecuta directamente sobre el software en funcionamiento validado por las suites de Katalon Studio y los reportes de SonarQube[2]. La Retrospectiva semanal se enfoca en mejoras incrementales de ingeniería, manteniendo discusiones ágiles y orientadas a la acción[3].

Plan de Ejecución de 10 Semanas (Hoja de Ruta Sprint por Sprint)

La distribución del trabajo a lo largo de las diez semanas está estructurada para mitigar riesgos arquitectónicos de manera temprana, consolidar el valor del negocio en la fase intermedia y dedicar la fase final al endurecimiento (*hardening*), pruebas de carga y despliegue final en producción.

Sprint 1: Fundaciones Arquitectónicas, Gobernanza y Configuración DevSecOps

El primer ciclo se enfoca en establecer la infraestructura de desarrollo, el pipeline de integración continua y la estructura base de la aplicación Java, junto con los tableros de gobierno en Notion[2]. Adolfo y Abel estructuran el proyecto base Java utilizando patrones de diseño por capas y la gestión de dependencias automatizada. Paralelamente, José despliega la instancia de SonarQube y configura los umbrales iniciales del *Quality Gate*[8]. Amir organiza la base de datos en Notion definiendo las propiedades de Criterios de Aceptación, estimación en puntos de historia y estados del flujo de trabajo[2]. Miguel crea los primeros scripts de prueba

funcional en Katalon Studio para la verificación automatizada del estado de los servicios[8]. El criterio de aceptación del Sprint exige la ejecución limpia del pipeline de integración sin vulnerabilidades estáticas detectadas por SonarQube[8].

Sprint 2: Autenticación, Autorización y Modelo de Dominio Base

En el segundo ciclo se implementa la gestión de identidades, la autenticación basada en tokens de seguridad (JWT/OAuth2) y el modelo de datos relacional de la aplicación[12]. Adolfo y Abel desarrollan los endpoints REST Java para el registro de usuarios, autenticación y gestión de sesiones. Miguel construye los casos de prueba automatizados en Katalon Studio para validar la emisión y revocación de tokens[8]. Durante este ciclo se ejecuta la primera prueba de seguridad dinámica utilizando sqlmap sobre la interfaz de inicio de sesión (sqlmap -u "http://staging/api/auth/login" --data="user=a&pass=b" --batch) para certificar que las consultas de persistencia sean inmunes a vectores de inyección[12]. La barrera de calidad de SonarQube se ajusta para exigir un mínimo de 80% de cobertura en pruebas unitarias Java en el módulo de seguridad[8].

Sprint 3: Módulo Core de Negocio I (Servicios y Registro de Datos)

El tercer ciclo aborda las interfaces de programación principales que habilitan las operaciones de negocio primarias del sistema. El equipo de desarrollo (Amir, José, Adolfo y Abel) implementa la lógica de procesamiento transaccional en Java, aplicando validaciones estrictas en la capa de entrada de datos. Miguel expande la suite de Katalon Studio para abarcar la validación de respuestas HTTP de la API REST[8]. Se realizan pruebas dinámicas con sqlmap variando los parámetros HTTP GET y POST para verificar la sanidad de los parámetros (sqlmap -u "http://staging/api/resource?id=1" --level=2)[12]. El entregable del Sprint se presenta al cliente patrocinador mediante la ejecución de transacciones reales verificadas en el entorno de staging[1].

Sprint 4: Módulo Core de Negocio II e Integración de Capa de Interfaz

Durante la cuarta semana se completan los flujos de trabajo avanzados del módulo de negocio y se integra la capa de interfaz de usuario con los servicios REST Java. El equipo de desarrollo entrega componentes de filtrado complejo, agregación de datos y generación de resultados[2]. Miguel automatiza flujos de prueba de extremo a extremo (*End-to-End*) en Katalon Studio, simulando la experiencia de usuario completa desde la interfaz gráfica hasta la persistencia en base de datos[8]. SonarQube analiza el código resultante para garantizar que la complejidad ciclomática del backend se mantenga bajo los límites establecidos[8]. El hito del Sprint consiste en la demostración de flujos de usuario completos operando sobre datos reales[1].

Sprint 5: Optimización de Persistencia y Evaluación Dinámica de Seguridad

La quinta semana se dedica a la optimización de la capa de persistencia Java Persistence API (JPA) y a la ejecución de una auditoría de seguridad dinámicamente profunda[12]. Adolfo y Abel

refactorizan las consultas de base de datos para eliminar cuellos de botella de rendimiento e implementar estrategias de indexación. Miguel ejecuta barridos exhaustivos con sqlmap inyectando encabezados de autorización y evaluando técnicas complejas de inyección (sqlmap -u "http://staging/api/v1/data" --headers="Authorization: Bearer <token>" --technique=BEUSTQ --batch --ignore-code 401)[12]. El hito del Sprint exige cero vulnerabilidades reportadas por sqlmap y la aprobación del reporte de rendimiento en SonarQube[8].

Sprint 6: Mapeo de Workflows Secundarios e Integraciones de Terceros

El sexto ciclo incorpora los componentes secundarios del sistema, incluyendo notificaciones asíncronas, exportación de reportes y consumo de APIs de terceros. Amir y José coordinan la integración de servicios externos asegurando la resiliencia del backend Java mediante patrones de tolerancia a fallos. Miguel amplía la suite de Katalon Studio para validar la recepción de respuestas externas y el comportamiento del sistema ante caídas simuladas de servicios[8]. SonarQube ejecuta un análisis de dependencias de software para identificar librerías de terceros con vulnerabilidades conocidas[8]. El entregable funcional se valida en la demostración semanal ante el cliente patrocinador[1].

Sprint 7: Pagos de Deuda Técnica y Automatización de Regresión

La séptima semana se consolida como un Sprint de refinamiento intensivo dedicado a saldar la deuda técnica acumulada, refactorizar módulos complejos y estabilizar la suite de pruebas[9]. Adolfo, Abel, Amir y José abordan la corrección de *Code Smells* y duplicaciones señaladas por SonarQube en Sprints anteriores[8]. Miguel reorganiza los scripts de Katalon Studio, optimizando los tiempos de ejecución de las pruebas automatizadas de regresión[8]. Este ciclo eleva el estándar del *Quality Gate* en SonarQube a un 85% de cobertura total de código, garantizando una base de software limpia y mantenible antes de la fase de endurecimiento final[8].

Sprint 8: Auditoría de Seguridad Profunda y Endurecimiento de la Plataforma

En el octavo ciclo se ejecuta una auditoría de seguridad avanzada sobre toda la plataforma Java[12]. El equipo aplica configuraciones de endurecimiento en los servidores de aplicación y activa cabeceras de seguridad estrictas en las respuestas HTTP. Miguel simula ataques ofensivos avanzados utilizando sqlmap con scripts *tamper* para evaluar el comportamiento del backend frente a cargas codificadas que intenten evadir controles (sqlmap -u "http://staging/api/v1/user" --cookie="sessionid=xyz" --level=5 --risk=3 --tamper=space2comment)[12]. El hito de este Sprint es la emisión de un certificado interno de seguridad que confirma la ausencia de vulnerabilidades de severidad alta o crítica tanto en SonarQube como en sqlmap[8].

Sprint 9: Pruebas de Aceptación de Usuario (UAT) y Ajustes de

Rendimiento

La novena semana se centra en desplegar el sistema completo en el entorno de pre-producción (*Staging*) para realizar las Pruebas de Aceptación de Usuario (UAT) directamente con el cliente patrocinador[2]. Amir actúa como facilitador de las sesiones de validación, registrando retroalimentación menor en Notion[2]. El equipo de desarrollo aplica correcciones cosméticas y optimizaciones de rendimiento de respuesta rápida. Miguel ejecuta de forma continua la suite de Katalon Studio como prueba de humo (*Smoke Test*) para garantizar que ningún ajuste de última hora introduzca regresiones en la plataforma[8]. El resultado es la aprobación formal de los flujos de negocio por parte del cliente patrocinador[2].

Sprint 10: Despliegue a Producción, Transferencia de Conocimiento y Cierre

El último ciclo ejecuta la migración final del producto software hacia el entorno de producción. Adolfo y José gestionan el despliegue de los artefactos Java y las bases de datos relacionales. Miguel ejecuta la suite final de verificación en Katalon Studio sobre el dominio de producción para certificar la operabilidad de la plataforma[8]. Amir y el equipo consolidan en Notion la documentación de arquitectura, el catálogo de APIs REST y los manuales de usuario. El proyecto concluye con la demostración final del producto en funcionamiento real ante el cliente patrocinador y la firma de entrega de los activos de software[2].

Sprint / Semana	Enfoque Principal	Herramientas Utilizadas	Hito Entregable al Patrocinador
Sprint 1	Arquitectura Base y CI/CD	Java, SonarQube, Notion, WhatsApp	Entorno de desarrollo operando y pipeline de CI configurado[2].
Sprint 2	Autenticación y Dominio	Java, Katalon Studio, sqlmap, SonarQube	Módulo de seguridad funcional e inicio de sesión verificado[8].
Sprint 3	Servicios Core API I	Java, Katalon Studio, SonarQube	APIs de procesamiento primario operativas en staging[8].
Sprint 4	Servicios Core API II	Java, Katalon	Flujo interactivo UI-Backend

	y UI	Studio, Notion	integrado y ejecutable[2].
Sprint 5	Optimización y Seguridad DAST	Java, sqlmap, SonarQube	Base de datos optimizada y API blindada contra SQLi[8].
Sprint 6	Workflows Secundarios	Java, Katalon Studio, SonarQube	Integraciones externas y motor de reportes funcional[8].
Sprint 7	Deuda Técnica y Regresión	SonarQube, Katalon Studio	Refactorización masiva y cobertura de pruebas ajustada[8].
Sprint 8	Hardening y Auditoría Dinámica	sqlmap, SonarQube	Certificación de seguridad estática y dinámica[8].
Sprint 9	Pruebas UAT y Tuning	Katalon Studio, Notion	Validación de usuario y aprobación formal del patrocinador[2].
Sprint 10	Producción y Cierre	Ecosistema completo	Producto en producción y entrega formal de activos[2].

Gobierno de Calidad, DevSecOps y Gestión de Riesgos

La sostenibilidad del proyecto durante las diez semanas depende de la automatización de la calidad y de una gestión de riesgos proactiva. El pipeline de DevSecOps opera como un flujo continuo donde cada fragmento de código sometido por los desarrolladores desencadena una secuencia automatizada de verificación[8].

El proceso inicia con la compilación del código Java en el servidor de integración continua, el cual envía los artefactos a SonarQube para realizar el análisis estático de seguridad y medir el cumplimiento de los *Quality Gates*[8]. Si SonarQube valida el código, este se despliega automáticamente en el entorno de staging. En esta fase, Katalon Studio ejecuta la suite de

pruebas funcionales E2E y de regresión de APIs, mientras que sqlmap efectúa escaneos dinámicos de penetración para descartar vulnerabilidades de inyección en las bases de datos[8]. El pipeline de calidad impone barreras estrictas: SonarQube falla la compilación si detecta cualquier vulnerabilidad clasificada como *Blocker* o *Critical*, o si la cobertura de código baja del 80%[8]. Asimismo, las pruebas de sqlmap se ejecutan bajo configuraciones CLI específicas que incluyen autenticación mediante tokens (sqlmap -u "http://staging/api/v1/secure" --headers="Authorization: Bearer <token>" --level=3 --risk=2) y técnicas de evasión con scripts tamper para garantizar la máxima cobertura de seguridad dinámicamente (sqlmap -u "http://staging/api/v1/login" --data="user=test&pass=test" --tamper=base64encode --ignore-code 401)[12].

Riesgo Identificado	**Impacto**	**Probabilidad**	**Estrategia de Mitigación**
Desviación de Alcance (*Scope Creep*) en Sprints cortos [cite: 3, 7]	Alto	Alta	Escudo protector del Scrum Master (José); congelamiento estricto del Sprint Backlog tras el Planning. Solicitudes nuevas ingresan al Product Backlog en Notion para priorización futura[2].
Sobrecarga de reuniones (*Meeting Overhead*) [cite: 16]	Medio	Alta	Cumplimiento estricto del *timeboxing* (Planning <= 90m, Daily <= 15m, Review <= 60m). Eliminación de reuniones no contempladas en el marco Scrum[2].
Conflicto de prioridades por rol dual (PO/Dev)	Alto	Media	Transparencia total en Notion. Las decisiones de

[cite: 5]			negocio de Amir deben estar respaldadas por criterios de aceptación claros verificables por el patrocinador antes de iniciar el desarrollo[2].
Aparición de vulnerabilidades críticas en la fase final [cite: 8, 12]	Crítico	Baja	Inclusión de escaneos de sqlmap y SonarQube desde el Sprint 2, evitando el descubrimiento tardío de fallos de arquitectura de seguridad[8].
Ausencia imprevista de un integrante del equipo [cite: 16]	Alto	Media	Mantenimiento de un búfer de capacidad del 15% en cada Sprint Planning. Dado que un día de ausencia reduce un 20% la capacidad individual, las tareas complejas se abordan mediante programación en parejas (*Pair Programming*) ocasional para compartir conocimiento del código[7].

Conclusiones y Recomendaciones Estratégicas

La ejecución de un proyecto de ingeniería de software estructurado en diez Sprints semanales

exige una disciplina operativa impecable, donde la agilidad no se confunde con la falta de rigor procesal[2]. El esquema presentado proporciona al cliente patrocinador la máxima transparencia operativa y financiera, garantizando que el capital invertido se traduzca semanalmente en un incremento de software funcional, verificado en calidad y blindado en seguridad[2].
Para asegurar el éxito sostenido de esta iniciativa, se destacan las siguientes recomendaciones clave:

- **Respetar la Inviolabilidad del Timebox y el Alcance del Sprint:** Una vez iniciado el Sprint el lunes por la mañana, el alcance acordado no debe ser alterado. Cualquier nueva necesidad detectada por el patrocinador o el Product Owner debe canalizarse a través del Product Backlog en Notion para ser priorizada en el siguiente ciclo semanal[2].
- **Automatización Completa como Eje No Negociable:** La velocidad exigida por Sprints de una semana hace inviable la ejecución manual de pruebas de regresión o auditorías de código. La suite automatizada de Katalon Studio y los Quality Gates de SonarQube deben mantenerse como el único criterio objetivo para declarar una historia como completada[2].
- **Mantenimiento Preventivo de la Deuda Técnica:** La inclusión explícita del Sprint 7 para la refactorización profunda asegura que la acumulación de pequeñas deudas técnicas no degrade la mantenibilidad del sistema a largo plazo[9]. Se debe evitar la tentación de sustituir este espacio de ingeniería por nuevas funcionalidades.
- **Disciplina en los Canales de Comunicación:** Mantener una separación estricta entre la documentación de requisitos en Notion y las alertas inmediatas en WhatsApp previene la pérdida de trazabilidad de los acuerdos de diseño y mantiene la alineación técnica de todo el equipo de desarrollo[2].

La combinación de una arquitectura Java sólida, la automatización del aseguramiento de calidad con SonarQube y Katalon Studio, la auditoría proactiva de seguridad mediante sqlmap y un gobierno Scrum transparente posiciona al equipo para entregar un producto comercial de alto impacto dentro del plazo estipulado de diez semanas[2].

Obras citadas

1. The 1-Week Sprint Dilemma! - Scrum.org, https://www.scrum.org/forum/scrum-forum/6713/1-week-sprint-dilemma
2. Scrum Sprint Project Management: A Detailed Agile Workflow Guide, https://teamhood.com/project-management/sprint-in-project-management/
3. What Is A Scrum Sprint? Key Steps And Best Practices For 2026, https://monday.com/blog/rnd/scrum-sprint/
4. Agile scrum roles and responsibilities - Atlassian, https://www.atlassian.com/agile/scrum/roles
5. Product Owner and Scrum Master Combined in One Individual?, https://www.scrum.org/resources/blog/product-owner-and-scrum-master-combined-one-individual
6. Agile Team Roles Product Owner & ScrumMaster, https://www.agilealliance.org/wp-content/uploads/2016/01/The-Dynamic-Duo-P

O-and-SM-Partnership-Agile-2012-bkadkins-ricksmith.pdf
7. Everything About Sprint Project Management. The Ultimate Guide, https://www.6sigma.us/project-management/sprint-project-management/
8. DevOps Implementation, Planning & Strategies - Sonar, https://www.sonarsource.com/resources/library/devops-implementation-guide/
9. Sprint Reviews in the AI Era, https://co-r-e.com/method/ai-sprint-review
10. SonarQube Tutorial for DevOps | Introduction to DevSecOps, https://www.youtube.com/watch?v=qyYsLVZDieU
11. How to Add SonarQube to Your Continuous Integration Process, https://solutionshub.epam.com/blog/post/sonarqube-to-the-continuous-integration-process
12. Usage · sqlmapproject/sqlmap Wiki - GitHub, https://github.com/sqlmapproject/sqlmap/wiki/usage
13. Sqlmap, the Tool for Detecting and Exploiting SQL Injections - Vaadata, https://www.vaadata.com/en/blog/sqlmap-the-tool-for-detecting-and-exploiting-sql-injections/
14. SQL Injection Attacks – How to Use SQLMap to Find Database, https://www.freecodecamp.org/news/how-to-protect-against-sql-injection-attacks/
15. Can sqlmap test for SQL injection in Basic Authentication?, https://security.stackexchange.com/questions/229581/can-sqlmap-test-for-sql-injection-in-basic-authentication
16. How to choose the optimal sprint length - LogRocket Blog, https://blog.logrocket.com/product-management/optimal-sprint-length/
17. Has anyone here done 1 week sprints, what was it like? - Reddit, https://www.reddit.com/r/agile/comments/hfktvl/has_anyone_here_done_1_week_sprints_what_was_it/
18. Integrating SonarCloud, Snyk & ZAP in a Java GitHub Action Pipeline, https://www.youtube.com/watch?v=F__XSbdzBgU
19. How to inject a part of cookie using sqlmap - Stack Overflow, https://stackoverflow.com/questions/24366856/how-to-inject-a-part-of-cookie-using-sqlmap
20. SQLMap Essentials - Hack The Dome, https://hackthedome.com/sqlmap-essentials/